Le code Morse : décoder sans ambiguïté ?

La calculatrice et les documents sont interdits. Une fonction demandée dans une question peut être utilisée dans les questions suivantes, même si elle n'a pas été écrite. Les questions peuvent être traitées dans l'ordre souhaité, à condition d'indiquer clairement leur numéro.

Rappels concernant le langage Python

- Les fonctions `min`, `max`, `sum`, `sort`, `sorted`, `index` et `count` sont **interdites**. Sont autorisés : `len(L)`, `L.append(e)`, `L.pop()`, le tranchage et la concaténation.
- **Dictionnaires.** On admet que l'accès, l'ajout et la modification d'une clé coûtent $O(1)$. Exemple d'utilisation :

```
d = {}                  # dictionnaire vide
d['E'] = '.'            # ajout de la clé 'E', associée à '.'
'E' in d                # True : la clé 'E' existe
for x in d:             # parcours des clés
    print(x, d[x])
```

- **Chaînes.** On admet que l'ajout d'un caractère à une chaîne (`s + x`, où `x` est un caractère) coûte $O(1)$.

Toute complexité demandée est une complexité temporelle dans le pire des cas, exprimée avec la notation O.

Barème indicatif : partie I sur 5 points, partie II sur 4 points, partie III sur 5,5 points, partie IV sur 5,5 points.

Introduction

Le code Morse. Inventé vers 1838 pour le télégraphe, le code Morse associe à chaque lettre une suite de signaux courts (**points**, notés `'.'`) et longs (**traits**, notés `'-'`). En général, les lettres fréquentes reçoivent des codes courts : la lettre E, la plus fréquente en anglais, est codée par un seul point.

Les pauses. Le message codé est la suite des codes de ses lettres. Pour qu'on sache où s'arrête chaque lettre, l'opérateur marque une **pause** entre deux lettres. Dans ce sujet, cette pause est représentée par un espace (le caractère `' '`) : le mot SOS se transmet `'... --- ...'`. On va voir que cette pause est indispensable.

L'arbre Morse. On représente le code par un arbre binaire :

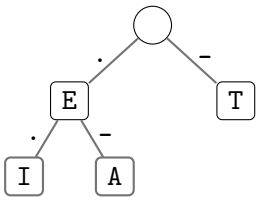
- la racine ne porte aucune lettre : son étiquette est `None` (aucune valeur en Python) ;
- **tous les autres nœuds**, internes comme feuilles, portent une lettre ;
- le code d'une lettre est le chemin de la racine à son nœud : on écrit `'.'` quand on descend à gauche et `'-'` quand on descend à droite.

En Python, un arbre non vide est la liste `[r, g, d]`, l'arbre vide est `[]`, et un nœud est une feuille si et seulement si `a[1] == []` et `a[2] == []`. Dans les arbres de ce sujet, chaque nœud a zéro ou deux fils.

Exemple : un petit arbre B à quatre lettres

```
B = [None, ['E', ['I', [], []],
                 ['A', [], []]],
           ['T', [], []]]
```

Pour trouver le code de A, on part de la racine, on descend à gauche (`'.'`) puis à droite (`'-'`) : le code de A est `'.-'`. La lettre E est sur un nœud *interne*.



codes : `E` → `.`, `I` → `..`, `A` → `.-`, `T` → `-`

Fil rouge. Dans tout le sujet, on utilise l'arbre Morse `M` réduit aux 14 lettres dont le code a au plus trois signaux :

```
M = [None, ['E', ['I', ['S', [], []], ['U', [], []]],
                 ['A', ['R', [], []], ['W', [], []]]],
           ['T', ['N', ['D', [], []], ['K', [], []]],
                 ['M', ['G', [], []], ['O', [], []]]]]
```

Plan du sujet.

- **Partie I :** lire l'arbre et construire la table des codes ;
- **Partie II :** coder et décoder un message *avec* séparateurs ;
- **Partie III :** montrer que le Morse n'est pas un code préfixe ;
- **Partie IV :** compter les façons de lire un message *sans* séparateur.

Partie I — Lire l'arbre

(5 points)

▶ **Q1 — Lecture de l'arbre.**

▶ **1.1** Dessiner l'arbre `M`, en indiquant `.` ou `-` sur chaque arête, comme dans l'exemple de l'arbre `B`.

▶ **1.2** Coder à la main le mot SOUK.

▶ **1.3** Décoder à la main la suite `'-- . -.. .. -. .-'`.

▶ **Q2 — Table des codes.** On veut construire le dictionnaire associant à chaque lettre son code. On utilise une fonction récursive `remplir(a, chemin, d)` qui ajoute au dictionnaire `d` le code de chaque lettre de l'arbre `a`, sachant que `chemin` est le code de la racine de `a`.

Attention : un nœud interne peut porter une lettre. Il faut donc enregistrer la lettre du nœud *puis*, s'il n'est pas une feuille, explorer ses deux fils.

▶ **2.1** Écrire la fonction `remplir(a, chemin, d)`. On n'oubliera pas que la racine de l'arbre complet porte `None`, qu'il ne faut pas enregistrer.

▶ **2.2** En déduire une fonction `table(a)` qui renvoie le dictionnaire des codes de l'arbre `a`.

Exemple

```
>>> table(B)
{'E': '.', 'I': '..', 'A': '.-', 'T': '-'}
```

▶ **2.3** Donner la complexité de `table(a)` en fonction du nombre N de nœuds de l'arbre `a`. Justifier.

Partie II — Coder et décoder avec séparateurs (4 points)

▶ **Q3 — Coder un mot.** Écrire une fonction `coder(m, d)` qui reçoit un mot `m` (une chaîne de lettres majuscules, sans espace) et un dictionnaire de codes `d`, et renvoie la chaîne Morse du mot, les codes étant séparés par **un seul** espace. Il ne doit pas y avoir d'espace à la fin.

Exemple

```
>>> coder('SOS', table(M))
'... --- ...'
```

▶ **Q4 — Décoder.**

▶ **4.1** Écrire une fonction `décoder_lettre(a, c)` qui renvoie la lettre de code `c` dans l'arbre `a`. On utilisera une variable `courant`, initialisée à `a`, qui désigne le nœud où l'on se trouve. On suppose que `c` est le code d'une lettre de l'arbre.

Exemple

```
>>> décoder_lettre(M, '-.-')
'K'
```

▶ **4.2** En utilisant `décoder_lettre`, écrire une fonction `décoder(a, s)` qui renvoie le mot codé par la chaîne `s` (codes séparés par un espace). On parcourra `s` caractère par caractère, en accumulant les signaux de la lettre en cours dans une chaîne `c`, que l'on décode à chaque espace. Attention à la dernière lettre, qui n'est suivie d'aucun espace.

Exemple

```
>>> décoder(M, '-.. --- ..- .- -. .')
'DOUANE'
```

▶ **4.3** Donner la complexité de `décoder` en fonction de la longueur n de `s`, en la justifiant.

Partie III — Le Morse n'est pas un code préfixe (5,5 points)

On dit qu'un code est **préfixe** si aucun code n'est le début d'un autre code.

▶ **Q5 — Sans les pauses.** Un télégraphiste distrait oublie toutes les pauses et transmet `'...'`.

▶ **5.1** Donner toutes les façons de lire ce message avec l'arbre `M`.

▶ **5.2** En utilisant l'arbre, expliquer pourquoi le code Morse n'est pas préfixe. Quelle propriété faudrait-il imposer aux nœuds portant une lettre pour que le code obtenu soit toujours préfixe ? Justifier.

▶ **Q6 — Test de préfixe.** Écrire une fonction `commence(s, i, c)` qui renvoie `True` si la chaîne `c` apparaît dans `s` à partir de l'indice `i`, c'est-à-dire si `s[i]`, `s[i+1]`, ..., `s[i+len(c)-1]` sont exactement les caractères de `c`, et `False` sinon. On n'utilisera **pas** le tranchage, et on veillera à ne jamais sortir de la chaîne `s`.

Exemple

```
>>> commence('..-.', 1, '.-')
True
>>> commence('..-.', 3, '.-')
False
```

En particulier, `commence(s, 0, c)` indique si `c` est un début de `s`.

▶ **Q7 — Un code est-il préfixe ?**

▶ **7.1** Écrire une fonction `code_préfixe(d)` qui reçoit un dictionnaire de codes `d` et renvoie `True` si le code est préfixe, `False` sinon. Les codes de `d` sont deux à deux distincts.

▶ **7.2** On note ℓ le nombre de lettres de `d` et L la longueur maximale d'un code. Donner la complexité de `code_préfixe` en fonction de ℓ et L.

▶ **7.3** Que renvoie `code_préfixe(table(M))` ? Justifier à l'aide de la question 5.

Partie IV — Combien de lectures possibles ? (5,5 points)

▶ **Q8 — Retirer les pauses et vérifier une lecture.**

▶ **8.1** Écrire une fonction `sans_pause(s)` qui renvoie la chaîne `s` sans ses espaces.

Exemple

```
>>> sans_pause('... --- ...')
'...---...'
```

Dans la suite, `d` est la table des codes de `M` et `s` est une chaîne de points et de traits **sans espace**. On appelle **décodage** de `s` toute façon de découper `s` en codes de lettres de `d`, écrits les uns après les autres.

▶ **8.2** Écrire une fonction `est_lecture(m, s, d)` qui renvoie `True` si le mot `m` est un décodage de `s`, c'est-à-dire si les codes des lettres de `m`, écrits les uns après les autres, donnent exactement `s`, et `False` sinon.

Exemple

```
>>> est_lecture('EI', '...', table(M))
True
>>> est_lecture('EA', '...', table(M))
False
```

▶ **Q9 — Une fonction récursive.** On note n_i le nombre de décodages de la fin de la chaîne `s`, à partir de l'indice `i`.

- Si `i == len(s)`, il ne reste rien à lire : il y a exactement un décodage (le mot vide).
- Sinon, pour chaque lettre `x` dont le code apparaît dans `s` à partir de l'indice `i`, on peut lire `x` puis décoder la suite à partir de l'indice `i + len(d[x])`. On fait la somme de ces nombres.

► **9.1** Recopier et compléter la fonction récursive `nb_décodages(s, d, i)` ci-dessous, qui renvoie n_i. Le nombre total de décodages de `s` est donc `nb_décodages(s, d, 0)`.

```
def nb_décodages(s, d, i):
    if _____________________ :
        return _______
    total = 0
    for x in d:
        if _____________________ :
            total = total + ___________________________
    return total
```

Exemple

```
>>> nb_décodages('...', table(M), 0)
4
>>> nb_décodages('...---...', table(M), 0)
149
```

► **9.2** On admet que, sur une chaîne de longueur n, `nb_décodages` effectue au moins $2^{\lfloor n/2 \rfloor}$ appels récursifs. Qu'en déduit-on sur sa complexité ?

► **Q10 — Mémoïsation.** Pour éviter de recalculer plusieurs fois n_i pour le même indice `i`, on mémorise les valeurs déjà calculées dans un dictionnaire `m` dont les clés sont des indices.

Écrire une fonction récursive `nb_mémo(s, d, i, m)` qui renvoie n_i en consultant `m` avant tout calcul et en y enregistrant chaque nouvelle valeur. L'appel initial se fait avec `i = 0` et `m = {}`.

Fin du devoir.